

Chapter 2 Entity-Relationship Modeling

2.1 Business Rules and Assumptions

Business Rules:

1. Member & Loyalty Relationships

- **Member and CustomerOrder**
 - Each member can place zero, one, or many orders, and each order must belong to one and only one member. (One to Many)
- **Member and PointRedemption**
 - Each member can make zero, one, or many point redemptions, and each point redemption must belong to one and only one member. (One to Many)
- **Voucher and PointRedemption**
 - Each voucher can be redeemed in zero, one, or many point redemptions, and each point redemption must be for one and only one voucher. (One to Many)
- **CustomerOrder and PointRedemption**
 - Each order can use zero, one, or many point redemptions, and each point redemption must be applied to one and only one order. (One to Many)

2. Branch & Employee Relationships

- **Branch and Employee**
 - Each branch can employ one or many employees, and each employee must work at one and only one branch. (One to Many)
- **Branch and CustomerOrder**
 - Each branch can process zero, one, or many orders, and each order must be processed at one and only one branch. (One to Many)

3. Inventory & Stock Management Relationships

- **Branch and Item (Stock)**
 - Each branch can hold stock zero, one, or many items, and each item can exist in stock across zero, one, or many branches. (Many to Many)

- **Stock to StockBatch**
 - Each stock record (an item at a specific branch) can consist of zero, one, or many stock batches (per stock batch), and each stock batch must belong to one and only one stock record. (One to Many)
- **StockLog to Stock and Employee**
 - Each stock log entry only belongs to one and only one stock and employee. Conversely, each stock and employee can be referenced in zero, one, or many stock log entries. (One to Many)

4. Supplier & Purchasing Relationships

- **Supplier and PurchaseOrder**
 - Each supplier can receive zero, one, or many purchase orders, and each purchase order must be issued to one and only one supplier. (One to Many)
- **PurchaseOrder and Item (PurchaseOrderItem)**
 - Each purchase order can consist of one or many items, and each item can exist in zero, one, or many purchase orders. (Many to Many)

5. Order Processing & Fulfillment Relationships

- **CustomerOrder and Item (OrderDetail)**
 - Each order can consist of one or many items, and each item can exist in zero, one, or many orders. (Many to Many)
- **CustomerOrder and Payment**
 - Each order can have zero or one payment, and each payment must belong to one and only one order. (One to One)
- **CustomerOrder and Delivery**
 - Each order can have zero or one delivery, and each delivery must belong to one and only one order. (One to One)
- **DeliveryService and Delivery**
 - Each delivery service can handle zero, one, or many deliveries, and each delivery must be assigned to one and only one delivery service. (One to Many)
- **CustomerOrder and Pickup**
 - Each order can have zero or one pickup, and each pickup must belong to one and only one order. (One to One)

6. Promotions, Refunds & Returns Relationships

- **Promotion and Item (PromotionItem)**
 - Each promotion can apply to one or many items, and each item can be included in zero, one, or many promotions. (Many to Many)
- **CustomerOrder and Refund**
 - Each order can have zero or one refund, and each refund must belong to one and only one order. (One to One)
- **Refund and Item (ReturnItem)**
 - Each refund can consist of one or many items, and each item can exist in zero, one, or many refunds. (Many to Many)

7. A customer must register for a Member account to earn points, or redeem vouchers. Guest checkouts are allowed but no points will be given, and will have to fill out the required form upon checkout.

8. Loyalty points are earned at a rate of 1 point for every RM 1.00 spent on completed orders.

9. Loyalty points expire 12 months from the date they are earned if left unredeemed.

10. Only one voucher can be applied per customer order.

11. The points earned can be used to redeem a supported item or vouchers.

12. Vouchers cannot be applied to orders where the pre-discount total is less than the voucher's minimum spend requirement.

13. VIP Member accounts have an extra benefit of discount for every item, and a free delivery voucher every month.

14. Member accounts with no order activity for 12 consecutive months are automatically set to 'Inactive' status. Used to notify and hook inactive accounts by providing welcome back offers.

15. Each Member account can be registered once per IC only.

16. Store business hours run from 9:00 AM to 10:00 PM daily, including public holidays.

17. Delivery service runs from 9:00 AM to 8:00 PM. Any delivery in public holidays will be postponed.

18. Stock reorder alerts are automatically triggered when the Quantity of an item at a specific branch falls at or below its defined ReorderLevel.

19. Expired or damaged goods must be removed from available inventory (and record the new quantity in stock database) and give an appropriate adjustment type (Damaged, Expired), or remark for special cases, recorded in StockLog.
20. Branch stock audits must be conducted monthly to explain the changes in quantity.
21. A PurchaseOrder status cannot be marked as 'Received' until all ordered line items are verified against the physical delivery at the branch.
22. Customers can choose either pickup or delivery for online order. Pickup order requires to select a branch (closest branch in GPS) and is free of additional charges, and delivery applies a fee depending on total items weight.
23. Allow up to 1 day to collect fresh consumables. Meaning if a customer orders on August 1, they must pick up before August 2's closing time.
24. If an order that is "Fresh Consumables" is not picked up within the designated period, the system automatically cancels the order, and will not refund to the customer to prevent wastage.
25. Allow up to 7 days to collect orders containing long-shelf life and non-perishable items to reduce pickup space.
26. If an order that is not fresh consumables is not collected within the designated period, the system automatically cancels the order, and will be refunded to the customer and charged with a small fee.
27. For long-shelf life and non-perishable order cancelled within 3 days, full refund without charge is issued, else a small fee increase each day will be applied. For fresh consumables cancelled within 2 hours can be refunded completely.
28. However, customers cannot cancel booking more than 3 times a day to prevent malicious holding of stock.
29. Customers must present a unique 6-digit PickupCode at the branch counter to claim an in-store pickup order.
30. Payments must be settled in full at the time of checkout via approved electronic payment channels (E-Wallet, Online Banking, Card, or Cash).
31. Return and refund requests for damaged, defective, or expired items must be submitted within 7 days of order delivery or pickup.

32. Approved refunds are credited back using the original payment method within 5 to 7 working days.

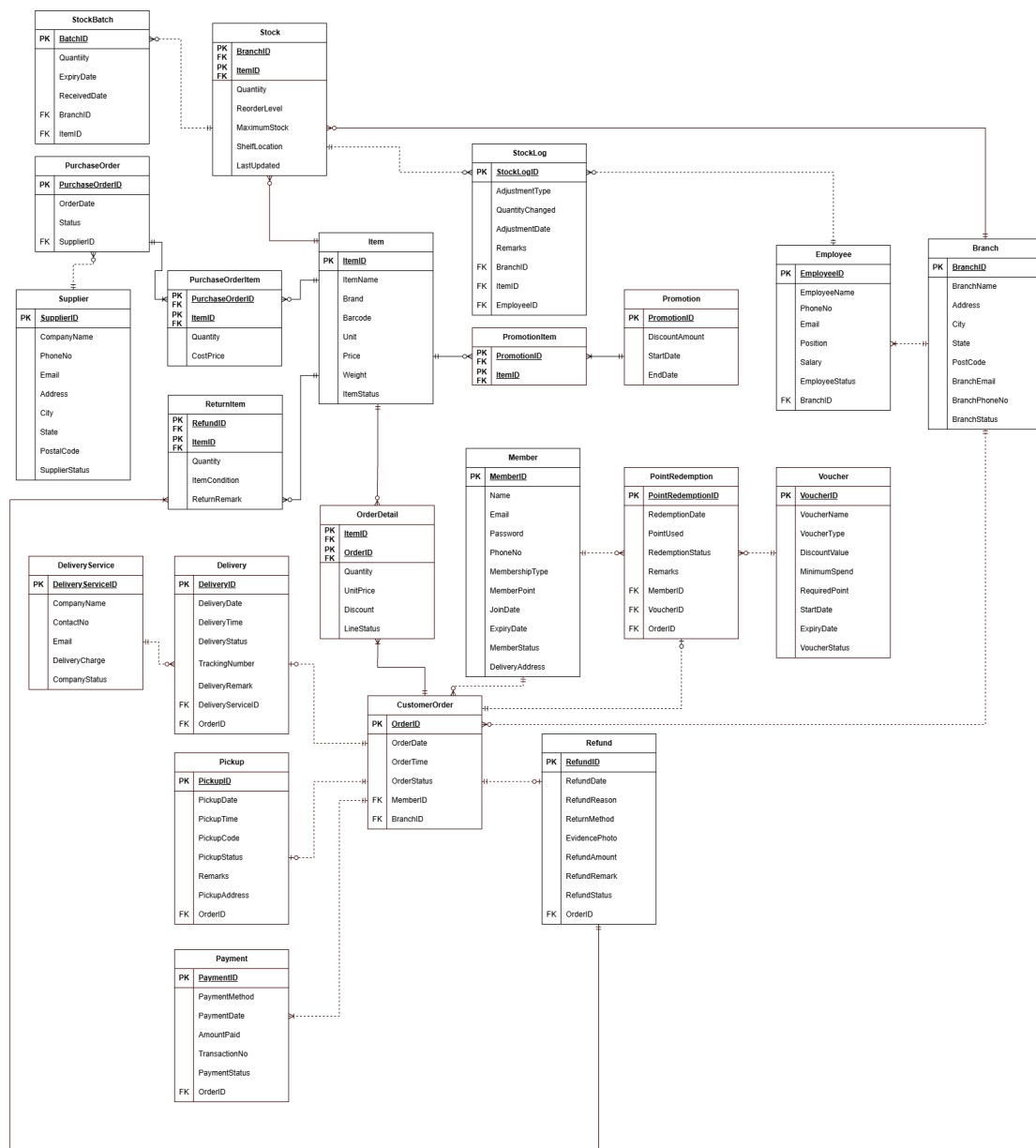
33. Any received items that are not damaged, defective, or expired cannot be refunded.

Assumptions:

- 1. All monetary transactions are processed strictly in Ringgit Malaysia (MYR).**
- 2. A customer order must select a single fulfillment type (either Pickup or Delivery, but not both simultaneously), split-shipment orders are not supported.**
- 3. System background tasks automatically evaluate and update order cancellations, member inactivity, and stock reorder alerts daily.**
- 4. National Registration Identity Card (NRIC/IC) numbers and 6-digit pickup codes are assumed to be unique per customer account and order session respectively.**
- 5. Online checkout payments via E-Wallet, Online Banking, Card, or Cash are settled instantly upon transaction authorization.**
- 6. Store business hours (9:00 AM – 10:00 PM) and delivery hours (9:00 AM – 8:00 PM) dictate the valid time frames for order collection.**

2.2 ERD

<https://drive.google.com/file/d/1MsQwYTKb8eatDCpiILDPltPAM85b7CCl/view>



Chapter 3 Data Definition

3.1 CREATE TABLE Statements

1. Member

```
CREATE TABLE Member (  
  MemberID      NUMBER(6)    NOT NULL,  
  Name          VARCHAR2(100) NOT NULL,  
  Email         VARCHAR2(100) NOT NULL,  
  Password      VARCHAR2(255) NOT NULL,  
  PhoneNo       VARCHAR2(20),  
  MembershipType VARCHAR2(20) NOT NULL,  
  MemberPoint   NUMBER(8)    DEFAULT 0 NOT NULL,  
  JoinDate      DATE         DEFAULT SYSDATE NOT NULL,  
  ExpiryDate    DATE,  
  MemberStatus  VARCHAR2(20) DEFAULT 'Active' NOT NULL,  
  DeliveryAddress VARCHAR2(200),  
  CONSTRAINT pk_member PRIMARY KEY (MemberID),  
  CONSTRAINT uq_member_email UNIQUE (Email),  
  CONSTRAINT ck_member_points CHECK (MemberPoint >= 0),  
  CONSTRAINT ck_member_type CHECK (MembershipType IN ('Normal','VIP')),  
  CONSTRAINT ck_member_status CHECK (MemberStatus IN ('Active','Inactive'))  
);
```

2. Branch

```
CREATE TABLE Branch (  
  BranchID      NUMBER(4)    NOT NULL,  
  BranchName    VARCHAR2(100) NOT NULL,  
  Address       VARCHAR2(200) NOT NULL,  
  City          VARCHAR2(50)  NOT NULL,  
  State         VARCHAR2(50)  NOT NULL,  
  PostCode      VARCHAR2(10),  
  BranchEmail   VARCHAR2(100),  
  BranchPhoneNo VARCHAR2(20),  
  BranchStatus  VARCHAR2(20) DEFAULT 'Active' NOT NULL,  
  CONSTRAINT pk_branch PRIMARY KEY (BranchID),  
  CONSTRAINT ck_branch_status CHECK (BranchStatus IN ('Active','Inactive'))  
);
```

3. Employee

```

CREATE TABLE Employee (
  EmployeeID      NUMBER(6)      NOT NULL,
  EmployeeName    VARCHAR2(100)  NOT NULL,
  PhoneNo        VARCHAR2(20),
  Email          VARCHAR2(100),
  Position       VARCHAR2(30)   NOT NULL,
  Salary         NUMBER(10,2),
  EmployeeStatus  VARCHAR2(20)   DEFAULT 'Active' NOT NULL,
  BranchID       NUMBER(4)      NOT NULL,
  CONSTRAINT pk_employee PRIMARY KEY (EmployeeID),
  CONSTRAINT fk_employee_branch FOREIGN KEY (BranchID)
    REFERENCES Branch(BranchID),
  CONSTRAINT ck_employee_salary CHECK (Salary >= 0),
  CONSTRAINT ck_employee_status CHECK (EmployeeStatus IN ('Active','Inactive'))
);

```

4. Item

```

CREATE TABLE Item (
  ItemID         NUMBER(6)      NOT NULL,
  ItemName       VARCHAR2(100)  NOT NULL,
  Brand          VARCHAR2(50),
  Barcode        VARCHAR2(50),
  Unit           VARCHAR2(20)   NOT NULL,
  Price          NUMBER(10,2)   NOT NULL,
  Weight         NUMBER(8,2),
  ItemStatus     VARCHAR2(20)   DEFAULT 'Available' NOT NULL,
  CONSTRAINT pk_item PRIMARY KEY (ItemID),
  CONSTRAINT uq_item_barcode UNIQUE (Barcode),
  CONSTRAINT ck_item_price CHECK (Price > 0),
  CONSTRAINT ck_item_status CHECK (ItemStatus IN ('Available','Unavailable'))
);

```

5. Stock

```

CREATE TABLE Stock (
  BranchID       NUMBER(4)      NOT NULL,
  ItemID         NUMBER(6)      NOT NULL,
  Quantity       NUMBER(8)      DEFAULT 0 NOT NULL,
  ReorderLevel   NUMBER(6)      DEFAULT 10 NOT NULL,
  MaximumStock   NUMBER(8)      NOT NULL,
  ShelfLocation  VARCHAR2(50),
  LastUpdated    DATE           DEFAULT SYSDATE NOT NULL,
  CONSTRAINT pk_stock PRIMARY KEY (BranchID, ItemID),

```

```

CONSTRAINT fk_stock_branch FOREIGN KEY (BranchID)
    REFERENCES Branch(BranchID),
CONSTRAINT fk_stock_item FOREIGN KEY (ItemID)
    REFERENCES Item(ItemID),
CONSTRAINT ck_stock_quantity CHECK (Quantity >= 0),
CONSTRAINT ck_stock_reorder CHECK (ReorderLevel >= 0),
CONSTRAINT ck_stock_maximum CHECK (MaximumStock >= 0)
);

```

6. StockBatch

```

CREATE TABLE StockBatch (
    BatchID      NUMBER(8)      NOT NULL,
    Quantity     NUMBER(8)      NOT NULL,
    ExpiryDate   DATE,
    ReceivedDate DATE          DEFAULT SYSDATE NOT NULL,
    BranchID     NUMBER(4)      NOT NULL,
    ItemID       NUMBER(6)      NOT NULL,
    CONSTRAINT pk_stock_batch PRIMARY KEY (BatchID),
    CONSTRAINT fk_batch_branch FOREIGN KEY (BranchID)
        REFERENCES Branch(BranchID),
    CONSTRAINT fk_batch_item FOREIGN KEY (ItemID)
        REFERENCES Item(ItemID),
    CONSTRAINT ck_batch_quantity CHECK (Quantity > 0),
    CONSTRAINT ck_batch_expiry CHECK (ExpiryDate IS NULL OR ExpiryDate >= ReceivedDate)
);

```

7. Supplier

```

CREATE TABLE Supplier (
    SupplierID   NUMBER(6)      NOT NULL,
    CompanyName  VARCHAR2(100) NOT NULL,
    PhoneNo      VARCHAR2(20),
    Email        VARCHAR2(100),
    Address      VARCHAR2(200),
    City         VARCHAR2(50),
    State        VARCHAR2(50),
    PostCode     VARCHAR2(10),
    SupplierStatus VARCHAR2(20) DEFAULT 'Active' NOT NULL,
    CONSTRAINT pk_supplier PRIMARY KEY (SupplierID),
    CONSTRAINT ck_supplier_status CHECK (SupplierStatus IN ('Active','Inactive'))
);

```

8. PurchaseOrder

```

CREATE TABLE PurchaseOrder (
  PurchaseOrderID  NUMBER(8)    NOT NULL,
  OrderDate        DATE          DEFAULT SYSDATE NOT NULL,
  Status           VARCHAR2(20)  DEFAULT 'Pending' NOT NULL,
  SupplierID       NUMBER(6)     NOT NULL,
  CONSTRAINT pk_purchase_order PRIMARY KEY (PurchaseOrderID),
  CONSTRAINT fk_po_supplier FOREIGN KEY (SupplierID)
    REFERENCES Supplier(SupplierID),
  CONSTRAINT ck_po_status CHECK (Status IN ('Pending','Approved','Received','Cancelled'))
);

```

9. PurchaseOrderItem

```

CREATE TABLE PurchaseOrderItem (
  PurchaseOrderID  NUMBER(8)    NOT NULL,
  ItemID           NUMBER(6)     NOT NULL,
  Quantity         NUMBER(8)     NOT NULL,
  CostPrice        NUMBER(10,2)  NOT NULL,
  CONSTRAINT pk_purchase_order_item PRIMARY KEY (PurchaseOrderID, ItemID),
  CONSTRAINT fk_poi_order FOREIGN KEY (PurchaseOrderID)
    REFERENCES PurchaseOrder(PurchaseOrderID),
  CONSTRAINT fk_poi_item FOREIGN KEY (ItemID)
    REFERENCES Item(ItemID),
  CONSTRAINT ck_poi_quantity CHECK (Quantity > 0),
  CONSTRAINT ck_poi_cost CHECK (CostPrice > 0)
);

```

10. StockLog

```

CREATE TABLE StockLog (
  StockLogID       NUMBER(8)     NOT NULL,
  AdjustmentType    VARCHAR2(30)  NOT NULL,
  QuantityChanged   NUMBER(8)     NOT NULL,
  AdjustmentDate    DATE          DEFAULT SYSDATE NOT NULL,
  Remarks           VARCHAR2(300),
  BranchID         NUMBER(4)     NOT NULL,
  ItemID           NUMBER(6)     NOT NULL,
  EmployeeID       NUMBER(6)     NOT NULL,
  CONSTRAINT pk_stock_log PRIMARY KEY (StockLogID),
  CONSTRAINT fk_stocklog_branch FOREIGN KEY (BranchID)
    REFERENCES Branch(BranchID),
  CONSTRAINT fk_stocklog_item FOREIGN KEY (ItemID)
    REFERENCES Item(ItemID),
  CONSTRAINT fk_stocklog_employee FOREIGN KEY (EmployeeID)

```

```
REFERENCES Employee(EmployeeID)
);
```

11. Promotion

```
CREATE TABLE Promotion (
  PromotionID    NUMBER(6)    NOT NULL,
  DiscountAmount NUMBER(10,2) NOT NULL,
  StartDate      DATE         NOT NULL,
  EndDate        DATE         NOT NULL,
  CONSTRAINT pk_promotion PRIMARY KEY (PromotionID),
  CONSTRAINT ck_promotion_discount CHECK (DiscountAmount >= 0),
  CONSTRAINT ck_promotion_date CHECK (EndDate > StartDate)
);
```

12. PromotionItem

```
CREATE TABLE PromotionItem (
  PromotionID    NUMBER(6)    NOT NULL,
  ItemID         NUMBER(6)    NOT NULL,
  CONSTRAINT pk_promotion_item PRIMARY KEY (PromotionID, ItemID),
  CONSTRAINT fk_promotionitem_promotion FOREIGN KEY (PromotionID)
    REFERENCES Promotion(PromotionID),
  CONSTRAINT fk_promotionitem_item FOREIGN KEY (ItemID)
    REFERENCES Item(ItemID)
);
```

13. Voucher

```
CREATE TABLE Voucher (
  VoucherID      NUMBER(6)    NOT NULL,
  VoucherName     VARCHAR2(100) NOT NULL,
  VoucherType     VARCHAR2(20) NOT NULL,
  DiscountValue   NUMBER(10,2) NOT NULL,
  MinimumSpend    NUMBER(10,2) DEFAULT 0 NOT NULL,
  RequiredPoint   NUMBER(8)    DEFAULT 0 NOT NULL,
  StartDate       DATE         NOT NULL,
  ExpiryDate      DATE         NOT NULL,
  VoucherStatus   VARCHAR2(20) DEFAULT 'Active' NOT NULL,
  CONSTRAINT pk_voucher PRIMARY KEY (VoucherID),
  CONSTRAINT ck_voucher_discount CHECK (DiscountValue >= 0),
  CONSTRAINT ck_voucher_spend CHECK (MinimumSpend >= 0),
  CONSTRAINT ck_voucher_point CHECK (RequiredPoint >= 0),
  CONSTRAINT ck_voucher_date CHECK (ExpiryDate > StartDate),
);
```

```
CONSTRAINT ck_voucher_status CHECK (VoucherStatus IN ('Active','Inactive','Expired'))
);
```

14. CustomerOrder

```
CREATE TABLE CustomerOrder (
  OrderID      NUMBER(8)      NOT NULL,
  OrderDate    DATE           DEFAULT SYSDATE NOT NULL,
  OrderTime    VARCHAR2(10),
  OrderStatus  VARCHAR2(20)   DEFAULT 'Pending' NOT NULL,
  MemberID     NUMBER(6)      NOT NULL,
  BranchID     NUMBER(4)      NOT NULL,
  CONSTRAINT pk_order PRIMARY KEY (OrderID),
  CONSTRAINT fk_order_member FOREIGN KEY (MemberID)
    REFERENCES Member(MemberID),
  CONSTRAINT fk_order_branch FOREIGN KEY (BranchID)
    REFERENCES Branch(BranchID),
  CONSTRAINT ck_order_status CHECK (OrderStatus IN ('Pending','Completed','Cancelled'))
);
```

15. OrderDetail

```
CREATE TABLE OrderDetail (
  ItemID       NUMBER(6)      NOT NULL,
  OrderID      NUMBER(8)      NOT NULL,
  Quantity     NUMBER(6)      NOT NULL,
  UnitPrice    NUMBER(10,2)   NOT NULL,
  Discount     NUMBER(10,2)   DEFAULT 0 NOT NULL,
  LineStatus   VARCHAR2(20)   DEFAULT 'Active' NOT NULL,
  CONSTRAINT pk_order_detail PRIMARY KEY (ItemID, OrderID),
  CONSTRAINT fk_orderdetail_item FOREIGN KEY (ItemID)
    REFERENCES Item(ItemID),
  CONSTRAINT fk_orderdetail_order FOREIGN KEY (OrderID)
    REFERENCES CustomerOrder(OrderID),
  CONSTRAINT ck_orderdetail_quantity CHECK (Quantity > 0),
  CONSTRAINT ck_orderdetail_price CHECK (UnitPrice > 0),
  CONSTRAINT ck_orderdetail_discount CHECK (Discount >= 0),
  CONSTRAINT ck_orderdetail_status CHECK (LineStatus IN ('Active','Cancelled','Returned'))
);
```

16. DeliveryService

```
CREATE TABLE DeliveryService (
  DeliveryServiceID NUMBER(6)      NOT NULL,
```

```

CompanyName    VARCHAR2(100) NOT NULL,
ContactNo      VARCHAR2(20),
Email          VARCHAR2(100),
DeliveryCharge  NUMBER(10,2) DEFAULT 0 NOT NULL,
CompanyStatus   VARCHAR2(20) DEFAULT 'Active' NOT NULL,
CONSTRAINT pk_delivery_service PRIMARY KEY (DeliveryServiceID),
CONSTRAINT ck_delivery_charge CHECK (DeliveryCharge >= 0),
CONSTRAINT ck_delivery_company_status CHECK (CompanyStatus IN ('Active','Inactive'))
);

```

17. Delivery

```

CREATE TABLE Delivery (
  DeliveryID      NUMBER(8) NOT NULL,
  DeliveryDate    DATE,
  DeliveryTime     VARCHAR2(10),
  DeliveryStatus  VARCHAR2(20) DEFAULT 'Pending' NOT NULL,
  TrackingNumber  VARCHAR2(100),
  DeliveryRemark  VARCHAR2(300),
  DeliveryServiceID NUMBER(6) NOT NULL,
  OrderID         NUMBER(8) NOT NULL,
  CONSTRAINT pk_delivery PRIMARY KEY (DeliveryID),
  CONSTRAINT fk_delivery_service FOREIGN KEY (DeliveryServiceID)
    REFERENCES DeliveryService(DeliveryServiceID),
  CONSTRAINT fk_delivery_order FOREIGN KEY (OrderID)
    REFERENCES CustomerOrder(OrderID),
  CONSTRAINT uq_delivery_tracking UNIQUE (TrackingNumber),
  CONSTRAINT uq_delivery_order UNIQUE (OrderID),
  CONSTRAINT ck_delivery_status CHECK (DeliveryStatus IN ('Pending','In
Transit','Delivered','Cancelled'))
);

```

18. Pickup

```

CREATE TABLE Pickup (
  PickupID       NUMBER(8) NOT NULL,
  PickupDate     DATE,
  PickupTime     VARCHAR2(10),
  PickupCode     VARCHAR2(50),
  PickupAddress  VARCHAR2(200),
  PickupStatus   VARCHAR2(20) DEFAULT 'Pending' NOT NULL,
  Remarks        VARCHAR2(300),
  OrderID        NUMBER(8) NOT NULL,
  CONSTRAINT pk_pickup PRIMARY KEY (PickupID),

```

```

CONSTRAINT fk_pickup_order FOREIGN KEY (OrderID)
REFERENCES CustomerOrder(OrderID),
CONSTRAINT uq_pickup_code UNIQUE (PickupCode),
CONSTRAINT uq_pickup_order UNIQUE (OrderID),
CONSTRAINT ck_pickup_status CHECK (PickupStatus IN
('Pending','Ready','Collected','Cancelled'))
);

```

19. Payment

```

CREATE TABLE Payment (
PaymentID      NUMBER(8)      NOT NULL,
PaymentMethod  VARCHAR2(20)   NOT NULL,
PaymentDate    DATE           DEFAULT SYSDATE NOT NULL,
AmountPaid     NUMBER(10,2)   NOT NULL,
TransactionNo  VARCHAR2(100),
PaymentStatus  VARCHAR2(20)   DEFAULT 'Pending' NOT NULL,
OrderID        NUMBER(8)      NOT NULL,
CONSTRAINT pk_payment PRIMARY KEY (PaymentID),
CONSTRAINT fk_payment_order FOREIGN KEY (OrderID)
REFERENCES CustomerOrder(OrderID),
CONSTRAINT uq_payment_transaction UNIQUE (TransactionNo),
CONSTRAINT ck_payment_method CHECK (PaymentMethod IN
('Cash','Card','E-Wallet','Online Banking')),
CONSTRAINT ck_payment_amount CHECK (AmountPaid >= 0),
CONSTRAINT ck_payment_status CHECK (PaymentStatus IN
('Pending','Paid','Failed','Refunded'))
);

```

20. PointRedemption

```

CREATE TABLE PointRedemption (
PointRedemptionID NUMBER(8)      NOT NULL,
RedemptionDate    DATE           DEFAULT SYSDATE NOT NULL,
PointUsed          NUMBER(8)      NOT NULL,
RedemptionStatus  VARCHAR2(20)   DEFAULT 'Pending' NOT NULL,
Remarks           VARCHAR2(300),
MemberID          NUMBER(6)      NOT NULL,
VoucherID         NUMBER(6)      NOT NULL,
OrderID           NUMBER(8)      NOT NULL,
CONSTRAINT pk_point_redemption PRIMARY KEY (PointRedemptionID),
CONSTRAINT fk_redemption_member FOREIGN KEY (MemberID)
REFERENCES Member(MemberID),
CONSTRAINT fk_redemption_voucher FOREIGN KEY (VoucherID)

```

```

REFERENCES Voucher(VoucherID),
CONSTRAINT fk_redemption_order FOREIGN KEY (OrderID)
REFERENCES CustomerOrder(OrderID),
CONSTRAINT ck_redemption_points CHECK (PointUsed > 0),
CONSTRAINT ck_redemption_status CHECK (RedemptionStatus IN
('Pending','Completed','Cancelled'))
);

```

21. Refund

```

CREATE TABLE Refund (
  RefundID      NUMBER(8)      NOT NULL,
  RefundDate    DATE           DEFAULT SYSDATE NOT NULL,
  RefundReason  VARCHAR2(300)  NOT NULL,
  ReturnMethod  VARCHAR2(30)   NOT NULL,
  EvidencePhoto VARCHAR2(300),
  RefundAmount  NUMBER(10,2)   NOT NULL,
  RefundRemark  VARCHAR2(300),
  RefundStatus  VARCHAR2(20)   DEFAULT 'Pending' NOT NULL,
  OrderID       NUMBER(8)      NOT NULL,
  CONSTRAINT pk_refund PRIMARY KEY (RefundID),
  CONSTRAINT fk_refund_order FOREIGN KEY (OrderID)
    REFERENCES CustomerOrder(OrderID),
  CONSTRAINT ck_refund_amount CHECK (RefundAmount >= 0),
  CONSTRAINT ck_refund_status CHECK (RefundStatus IN
('Pending','Approved','Rejected','Completed'))
);

```

22. ReturnItem

```

CREATE TABLE ReturnItem (
  RefundID      NUMBER(8)      NOT NULL,
  ItemID        NUMBER(6)      NOT NULL,
  Quantity      NUMBER(6)      NOT NULL,
  ItemCondition  VARCHAR2(30)   NOT NULL,
  ReturnRemark   VARCHAR2(300),
  CONSTRAINT pk_return_item PRIMARY KEY (RefundID, ItemID),
  CONSTRAINT fk_returnitem_refund FOREIGN KEY (RefundID)
    REFERENCES Refund(RefundID),
  CONSTRAINT fk_returnitem_item FOREIGN KEY (ItemID)
    REFERENCES Item(ItemID),
  CONSTRAINT ck_returnitem_quantity CHECK (Quantity > 0),
  CONSTRAINT ck_returnitem_condition CHECK
    (ItemCondition IN ('New','Good','Damaged','Defective'))
);

```

);